PAGED-TIERKV: COMPACT TIERED KV CACHE MANAGEMENT FOR HIGH-CONCURRENCY LLM INFERENCE

Martin Kang [1] **Jingwu Wang** [2]

ABSTRACT

Long-context language-model serving is increasingly limited by the memory footprint and bandwidth cost of the key-value (KV) cache. We built Paged-TierKV, a block-level KV-cache manager that routes historical KV blocks into HOT full-precision storage, WARM int8 storage, or COLD metadata-only storage while preserving a paged logical view for attention. The system combines a C++ block manager, a Python policy/runtime layer, a compact state-separated tensor pool, and a decode-only Triton attention kernel. The central systems question is whether logical KV compression can become real physical memory savings without reintroducing full-cache reconstruction or destroying long-context quality. On TinyLlama with 1900-token prompts under high request concurrency, Paged-TierKV reduces the memory growth rate from roughly 43 MB/request for the dense baseline to roughly 27 MB/request near the boundary, and it succeeds at 512 concurrent requests where the dense baseline runs out of memory. The system does not outperform the dense baseline in raw throughput; instead, its strongest result is converting logical KV compression into physical memory savings while maintaining usable quality and throughput under high KV pressure.

1 INTRODUCTION

Autoregressive large language model (LLM) inference stores a key and value tensor for every generated or prefetched token in every decoder layer. This KV cache avoids recomputing past activations, but its footprint grows linearly with context length, number of layers, number of KV heads, head dimension, and number of active requests. More concretely, the dense fp16 KV footprint is proportional to $2 \cdot R \cdot L \cdot T \cdot H_{kv} \cdot D \cdot 2$ bytes, where R is the number of resident requests, L is the layer count, T is context length, H_{kv} is the number of KV heads, and D is head dimension. Model weights are fixed after loading, but this cache grows with workload pressure. In long-context or high-concurrency serving, the KV cache can therefore become the dominant incremental working set.

Paged-TierKV targets this systems bottleneck. The project asks whether a paged, block-level cache can preserve the important parts of history in high precision, compress less important history, and expose real memory savings without reconstructing the full KV cache on every decode step. This distinction between logical and physical savings is important. A policy may mark most blocks as compressed, but if the implementation still allocates dense HOT and WARM capacity for every physical block, peak GPU memory will not improve. Earlier versions of the system demonstrated the logical idea but were limited by Python-side reconstruction and dense pool allocation. The final system uses a compact state-separated pool and Triton decode path so that HOT blocks consume fp16 storage, WARM blocks consume int8 storage plus metadata, and COLD blocks consume no dense KV storage.

Our main contribution is an end-to-end prototype integrated into a Hugging Face LLaMA-style model path. It implements block allocation, attention-based policy enforcement, compact state migration, decode-time fused dequantization/attention, and memory-realism telemetry. The evaluation shows a memory-scaling advantage under high KV pressure: in the main TinyLlama experiment, the dense baseline succeeds up to 448 concurrent 1900-token requests and fails at 512, while Paged-TierKV succeeds at 512. The result is not a universal throughput win. TierKV pays overhead for WARM dequantization, metadata indirection, and request-local decode scheduling. Its strongest validated claim is that compact tiered KV storage lowers the memory growth slope and extends the supported load before out-of-memory. We also run a 7B MHA-style Vicuna extension, which strengthens the memory-scaling story but does not produce a throughput crossover.

[1]Information Networking Institute, Carnegie Mellon University, Pittsburgh, PA, USA [2]School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, USA. Correspondence to: Martin Kang <lixuank@andrew.cmu.edu>, Jingwu Wang <jingwuwa@andrew.cmu.edu>.

Proceedings of the 8^{th} MLSys Conference, Santa Clara, CA, USA, 2025.

2 PROBLEM

The dense KV-cache baseline stores every historical token in fp16 until the request ends. This is simple and fast: attention reads a contiguous representation with no per-block state checks, no dequantization, and no policy bookkeeping. The drawback is that memory grows with total active context. If the serving system holds many concurrent long-context requests, dense KV quickly pushes the GPU toward its memory boundary even if the model weights themselves fit comfortably.

Two straightforward alternatives are insufficient on their own. Uniform quantization can reduce cache bytes, but it applies the same approximation to recent and important history and can introduce dequantization overhead on every attention read. Pure sparsification or token dropping saves more memory, but it can destroy retrieval or QA behavior when the removed blocks contain the answer. Our Qasper results demonstrate both sides: pure quantization preserves most quality but is slow, while pure sparsification has high compression and higher throughput but collapses F1.

Paged-TierKV addresses the following systems question: can we combine paged metadata, mixed precision, and selective eviction so that the cache uses less physical memory while preserving enough context for long-context QA? The central tradeoff is three-way. HOT fp16 blocks maximize fidelity and simple reads but consume dense memory. WARM int8 blocks retain information at lower memory cost but require dequantization during attention. COLD blocks remove resident KV storage entirely but lose the represented content. A useful system must therefore track block state cheaply, migrate blocks safely, avoid full-cache reconstruction in the decode hot path, and expose truthful memory metrics. The project is successful only if compression affects actual allocated GPU memory, not just a logical accounting table.

3 RELATED WORK

Paged KV-cache serving systems such as paged attention (Kwon et al., 2023) decouple logical sequence layout from physical GPU memory allocation. They motivate this project because they show that block tables are a practical interface between model-level attention and allocator-level memory management. Paged-TierKV uses the same broad idea of a logical block table, but adds state-dependent storage tiers and compression rather than only paging dense KV blocks.

KV-cache quantization reduces the byte footprint of historical keys and values (Zirui Liu et al., 2023; Sheng et al., 2023). Our WARM tier follows this direction, using int8 storage with scale and zero-point metadata. The project differs from uniform quantization by reserving a small set of HOT blocks in fp16, selected by attention-derived policy, so important recent or high-attention blocks need not pay quantization error.

KV sparsification and token eviction reduce memory by dropping parts of the context (Zhang et al., 2023; Xiao et al., 2023). Our COLD tier is a controlled version of this idea. The evaluation confirms the risk: pure sparsification achieves low resident KV but Qasper F1 collapses, while HOT+WARM keeps F1 close to the dense baseline.

Long-context benchmarks and LLM serving frameworks provide the workload context for this project. We use a Qasper subset from LongBench (Bai et al., 2024) for quality and staged synthetic concurrency to drive KV pressure, testing on models such as TinyLlama (Zhang et al., 2024) and Vicuna (Chiang et al., 2023). The implementation relies on PyTorch (Paszke et al., 2019), Hugging Face Transformers (Wolf et al., 2020), Triton (Tillet et al., 2019), and Modal (Modal Labs, 2024) for GPU execution.

4 OVERVIEW

Paged-TierKV treats each layer's KV history as a sequence of fixed-size blocks. Each logical block is assigned one of three states. HOT blocks are stored in fp16 and are used for attention without quantization. WARM blocks are stored in uint8 with per-token metadata and dequantized inside the decode attention kernel. COLD blocks keep metadata and valid length but no resident dense KV storage.

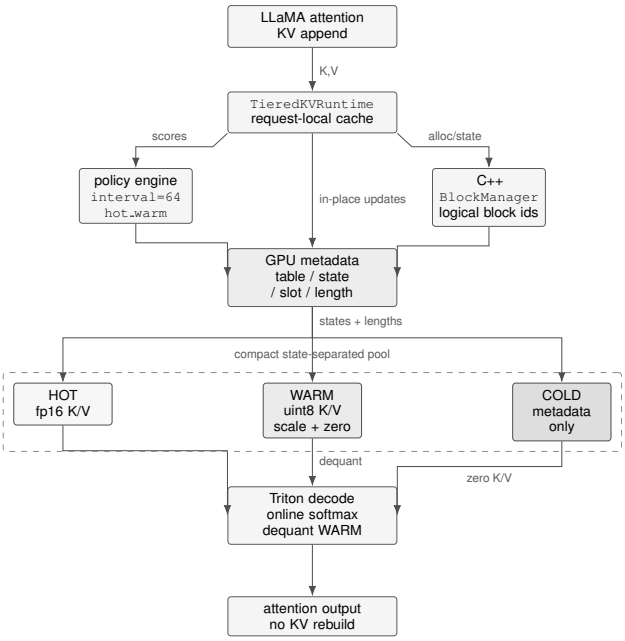

Figure 1. Paged-TierKV system overview.

The default configuration used in the final results is `policy_interval=64`,

`policy_on_new_block=False`, and `policy_mode=hot_warm`. Prefill remains eager and computes a policy signal once for the prompt. Decode uses a Triton kernel for query length one. The final evaluation separates the main TinyLlama high-concurrency result from a supporting 7B Vicuna experiment, because the extension tests a different question: whether a larger MHA model creates enough KV pressure for a throughput crossover.

5 System Design

5.1 Block Metadata and Runtime

The C++ `BlockManager` is the metadata source for block allocation and state transitions. It gives each logical KV block a stable identity and tracks whether that block is HOT, WARM, or COLD. The Python `TieredKVRuntime` owns the model-facing cache object, policy engine, tensor pool, layer state, profiling counters, and memory telemetry. This split keeps the allocator/policy interface explicit while allowing the Hugging Face model path to treat TierKV as a cache implementation.

Each layer has a logical table of active blocks, block states, slot indices, and block lengths. The block table preserves logical order. The state table selects the HOT, WARM, or COLD read path. The slot-index table maps a logical block to its current state-specific storage slot. The length table masks unused positions in the final partial block. This metadata stays resident on GPU for the decode path. The attention layer prefers the runtime attached to `past_key_values`, which allows multiple active TierKV caches to coexist. In the staged benchmark, concurrency is implemented as many independent batch-1 requests rather than true batched attention; this preserves the current Triton contract while creating real aggregate KV pressure.

5.2 Compact State-Separated Pool

The final memory result depends on separating logical block identity from physical storage slots. A dense tiered pool can report many WARM or COLD blocks but still allocate enough tensor capacity for every block to be HOT or WARM. That design shows logical compression but not physical savings. Paged-TierKV's compact pool instead allocates independent HOT and WARM storage. HOT slots store fp16 keys and values. WARM slots store uint8 keys and values plus fp16 scale/zero metadata. COLD blocks store no dense KV tensor.

State transitions migrate slots explicitly. A new decode block allocates a HOT slot. HOT-to-WARM migration quantizes from the HOT slot into a WARM slot and frees the HOT slot. WARM-to-HOT migration, when used by the policy, allocates a HOT slot, dequantizes, and frees the WARM slot. HOT/WARM-to-COLD frees any dense slot and keeps only metadata. COLD promotion is not supported in the current policy, matching the irreversible COLD semantics used in the implementation. This state-separated layout is the key change that converts the policy's logical compression into reduced physical KV allocation.

5.3 Policy Scheduling

The policy protects sink/recent blocks and selects additional HOT blocks from attention-derived block scores. The default HOT+WARM mode keeps selected blocks HOT and demotes all other live blocks to WARM. HOT+WARM+COLD additionally uses a WARM budget and demotes remaining blocks to COLD, which improves compression but hurts quality. The policy is deliberately not run on every new block in the final configuration; `policy_on_new_block=False` lets `policy_interval` control the update cadence. Prior profiling showed policy-path work, not the Triton decode kernel alone, was a main bottleneck. The final `policy_interval=64` setting therefore amortizes scoring and migration while keeping the block state stable enough to preserve Qasper quality.

5.4 Triton Decode Path

For decode, where query length is one, the model calls `tierkv_decode_attention`. The kernel launches over query heads, maps each query head to a KV head, iterates over logical blocks, and reads the state and slot index for each block. HOT loads fp16 K/V directly. WARM loads uint8 K/V and dequantizes inside the kernel. COLD contributes zero K/V while preserving the valid sequence positions in the softmax denominator. This avoids Python-side `torch.cat` reconstruction in the decode hot path, so sequence-length-dependent work is pushed into the GPU kernel rather than repeated Python tensor construction. The implementation remains decode-only and batch-1 per request; the multi-request benchmark scales pressure by keeping many independent caches resident.

5.5 Telemetry

The runtime reports active HOT/WARM/COLD block counts, logical resident KV, dense-equivalent KV, physical KV allocation, compression ratios, and pool utilization. Dense-equivalent KV estimates how much fp16 KV the active blocks would consume with no compression. Logical resident KV counts the represented state: fp16 bytes for HOT, int8 bytes plus metadata for WARM, and zero dense bytes for COLD. Physical KV allocation counts actual allocated HOT/WARM pools using tensor sizes. Peak memory is measured with `torch.cuda.max_memory_allocated()` in clean

runs. These counters are important because early dense-pool versions showed logical compression without corresponding physical savings. The final tables report both logical and physical compression to distinguish policy effectiveness from allocator effectiveness.

6 Evaluation

6.1 Setup

All final experiments run on Modal with an NVIDIA A10G. The main model is TinyLlama-1.1B-Chat with legal 1900-token inputs, leaving room under the 2048-token context limit for generation. Quality uses 50 Qasper examples from LongBench, left-truncated to 1900 input tokens and decoded for short answers. Throughput scaling uses decode-only timing after prefill, so the high-concurrency curves measure the incremental decode regime where KV pressure matters most. The high-pressure benchmark holds multiple independent batch-1 requests resident and decodes them round-robin.

We compare a dense Hugging Face baseline, pure quantization, pure sparsification, and Paged-TierKV. The primary Paged-TierKV configuration is HOT+WARM with hot budget 8. Memory tables use peak allocated memory, not reserved memory, and report TierKV's logical and physical KV counters before cache cleanup. The supporting extension uses `lmsys/vicuna-7b-v1.5`, a LLaMA-compatible MHA model with hidden size 4096, 32 layers, 32 attention heads, 32 KV heads, and max position embeddings 4096.

6.2 Quality and Compression

Table 1 shows that HOT+WARM Paged-TierKV preserves most of the baseline Qasper F1, while pure sparsification collapses. The budget-8 configuration reaches 10.67 F1 versus 11.17 for the dense baseline, a small absolute loss relative to the failure mode of pure sparsification at 0.93 F1. This is the quality argument for keeping most non-HOT blocks WARM rather than COLD. Its physical compression is 1.68× on the Qasper run, showing that the compact pool turns WARM placement into actual lower KV allocation rather than only a logical label.

The tier-mode ablation in Table 2 clarifies the quality-memory tradeoff. HOT-only preserves baseline quality but offers no compression: its physical compression is 0.99×, effectively dense storage. HOT+WARM keeps quality close to baseline and gives meaningful physical compression. HOT+WARM+COLD improves physical compression to 4.40× but reduces Qasper F1 to 1.47, so it is not the quality-preserving default. This result is useful because it separates two claims. TierKV can aggressively reduce memory with COLD blocks, but the practical default must keep enough WARM information to answer long-context questions.

Table 1. Qasper-50 quality and memory summary at 1900 input tokens.

Configuration	Peak MB	F1	tok/s	Phys. comp.
Baseline	2238.56	11.17	53.29	–
Pure Quantization	2220.81	10.75	16.27	1.75×
Pure Sparsification	2198.53	0.93	32.94	29.99×
Paged-TierKV, budget 4	2220.46	10.55	16.44	1.77×
Paged-TierKV, budget 8	2222.87	10.67	16.75	1.68×

Table 2. Tier-mode ablation. HOT+WARM is the final default because it preserves quality while compressing physical KV.

Mode	F1	tok/s	HOT/WARM/COLD	Phys. comp.
HOT-only	11.16	23.53	2662/0/0	0.99×
HOT+WARM	10.67	12.77	220/2442/0	1.68×
HOT+WARM+COLD	1.47	19.80	220/704/1738	4.40×

The distinction between logical and physical compression matters throughout the evaluation. Pure sparsification reports very high compression because almost all non-protected blocks become COLD, but the quality loss makes that point unusable. HOT+WARM has a lower compression factor, around 1.68× physical compression on Qasper, but it preserves the semantic information needed by the QA task. This is the operating point used for the high-pressure scaling results.

6.3 High-Concurrency Scaling

The main systems result is the staged TinyLlama concurrency benchmark. Table 3 compresses the curve to representative points. TierKV's peak memory grows more slowly than the dense baseline. The memory gap grows from 529.50 MB at 32 requests to 7239.73 MB at 448 requests. In the boundary run, the dense baseline succeeds at 448 requests but fails at 512 with CUDA out of memory. Paged-TierKV succeeds at 512 requests with 15159.07 MB peak memory, 12952.00 MB physical KV allocation, and 1.65× physical KV compression.

Table 4 summarizes the memory slopes printed by the staged runner. The dense baseline grows at roughly 42–43 MB/request across TinyLlama stages. TierKV grows at roughly 25–27 MB/request. This is the clearest evidence that the compact pool changes the scaling curve, not just one high-load point. The 7B extension shows the same pattern at a larger per-request scale: baseline memory grows faster than TierKV at both 1024-token and 1536-token contexts.

Table 3. Representative TinyLlama high-concurrency scaling points with 1900-token prompts. Throughput ratio is TierKV divided by dense baseline.

Requests	Baseline MB	TierKV MB	TierKV phys. KV MB	Dense-eq. KV MB	Throughput ratio	Status
32	3439.77	2910.27	792.00	1317.94	0.88×	both OK
160	8823.58	6180.70	4037.00	6699.69	0.80×	both OK
320	15536.85	10267.70	8081.00	13410.38	0.68×	both OK
448	20711.66	13471.93	11256.00	18583.12	0.88×	both OK
512	OOM	15159.07	12952.00	21373.00	–	baseline OOM

Table 4. Peak-memory slope summary. Lower MB/request means better memory scaling as concurrency grows.

Range	Baseline	TierKV
TinyLlama 32–160	42.06	25.55
TinyLlama 192–320	43.01	26.08
TinyLlama 384–448	43.26	26.90
Vicuna 1024, 1–12	536.00	339.63
Vicuna 1536, 1–8	791.65	479.32

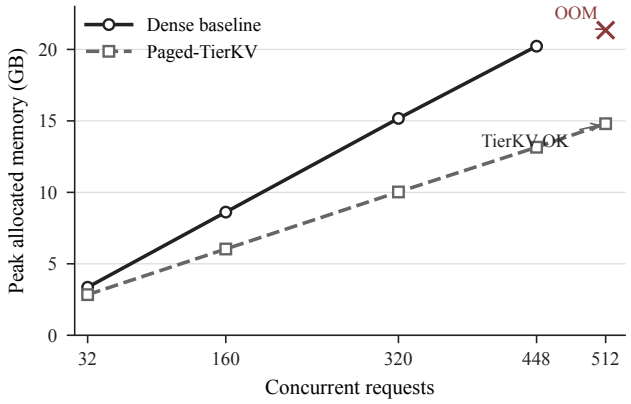

Figure 2. Peak memory scaling for the TinyLlama high-concurrency benchmark.

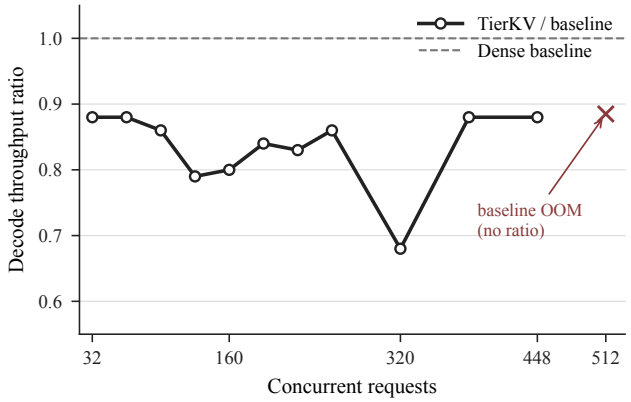

Figure 3. Throughput ratio for the TinyLlama high-concurrency benchmark.

The boundary summary is shown in Table 5. This is the strongest result in the project: the compact state-separated pool converts logical compression into physical memory savings large enough to extend the maximum supported concurrent load. It also clarifies the role of throughput. TierKV is not faster than dense baseline at matched successful points, but the baseline cannot produce a 512-request throughput number because it fails before the measurement completes.

Table 5. Failure boundary in the final TinyLlama staged sweep.

Method	Max OK req.	Total ctx. tokens	Failure
Baseline	448	851200	OOM at 512
Paged-TierKV	512	972800	none observed

6.4 7B Extension

The 7B extension tests whether a larger MHA-style model becomes KV-bound enough for TierKV to exceed dense throughput. It does not. However, the extension supports the memory-scaling story on a model with much larger per-request KV footprint. The passkey sanity check confirms both baseline and TierKV preserve the expected passkey. At 1024-token context, the TierKV/dense throughput ratio improves from 0.44× to 0.77× as requests increase from 1 to 12, while the memory gap grows from 169.65 MB to 2329.80 MB. At 1536-token context, the ratio improves from 0.45× to 0.75×, while the memory gap grows from 301.42 MB to 2487.73 MB.

Table 6. Vicuna-7B MHA extension summary. No throughput crossover is observed.

Context	Req. range	Ratio trend	Gap trend MB	Last comp.
1024	1–12	0.44–0.77×	169.65–2329.80	1.60×
1536	1–8	0.45–0.75×	301.42–2487.73	1.70×

This extension should be read as supporting evidence rather than the main result. It uses a different model and lower request counts because a 7B model has much higher base

memory. The useful observation is that the same slope pattern appears: TierKV memory grows more slowly as concurrency increases. The throughput gap narrows with pressure, but it does not close. That means the current implementation's extra WARM-path and metadata overhead still dominates before a raw throughput crossover appears.

6.5 Overhead Analysis

While Paged-TierKV significantly reduces physical memory allocation, this compression inherently introduces overhead across five main dimensions.

First, maintaining GPU-resident metadata—including logical block tables, block states, slot indices, and valid lengths—consumes device memory. Although this metadata incurs a minor storage cost, it is a necessary tradeoff that successfully eliminates CPU-GPU synchronization and removes Python-side cache reconstruction from the decode hot path.

Second, WARM dequantization introduces additional computational and memory bandwidth costs. For blocks stored in uint8, the decode kernel must load the compressed keys and values alongside their scale and zero-point metadata to reconstruct the tensors on the fly. This process requires more arithmetic operations and memory traffic than directly reading a dense fp16 cache.

Third, metadata indirection disrupts contiguous memory access patterns. Instead of reading a single continuous fp16 layout, the decoding phase must first consult block states and slot metadata to route reads to HOT, WARM, or COLD storage. While this lookup cost is minimal per block, it accumulates across every single decode step.

Fourth, policy bookkeeping requires active runtime management. The system must track attention-derived scores, update tier assignments, and handle slot migrations within the compact pool. Even though policy updates are amortized (e.g., via `policy_interval=64`), the associated state machine and migration logic still impose control overhead that is entirely absent in the dense baseline.

Finally, per-request runtime management limits throughput scaling under high load. Because the staged benchmark maintains a request-local runtime for each active sequence rather than utilizing a fused batched kernel, request metadata, cache states, and scheduling are processed independently. This design choice ensures compatibility with the current batch-1 Triton path, but the resulting control overhead scales linearly as concurrency increases.

6.6 Discussion

Paged-TierKV does not make decode universally faster than the dense baseline. Its strongest result is instead a memory-scaling win: under high KV pressure, the compact pool lowers physical memory growth and extends the supported load. The current implementation still pays for WARM dequantization, state-aware routing, and request-local control overhead, so future work should focus on reducing WARM-path cost and integrating more fused batched serving.

7 Conclusion

We built Paged-TierKV, an end-to-end tiered KV-cache prototype with paged metadata, attention-based policy, compact state-separated storage, Triton decode, and live memory telemetry. The final system shows that logical KV compression can become real physical memory savings when HOT, WARM, and COLD blocks occupy state-specific storage rather than dense all-state capacity. On TinyLlama high-concurrency inference, Paged-TierKV supports 512 concurrent 1900-token requests while the dense baseline runs out of memory at that point. The 7B Vicuna extension shows the same memory-slope advantage under a larger MHA model, but still no throughput crossover. The remaining limitation is throughput: TierKV stays below dense baseline speed in the tested successful regimes because compressed storage introduces dequantization and metadata overhead. The broader takeaway is that tiered KV management is most compelling as a memory-scaling mechanism under high concurrency, and future work should focus on reducing WARM-path overhead, fusing more metadata work, and integrating true batched serving.

References

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A bilingual, multitask benchmark for long context understanding. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL https://aclanthology.org/2024.acl-long.172/.

Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL https://lmsys.org/blog/2023-03-30-vicuna/.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.

Modal Labs. Modal: Fast, serverless compute for AI. https://modal.com/, 2024.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pp. 8024–8035. Curran Associates, Inc., 2019.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Re, C., Stoica, I., and Zhang, C. FlexGen: High-throughput generative inference of large language models with a single GPU. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 31094–31116. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/sheng23a.html.

Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020.

Xiao, G., Tian, Y., Chen, B., Song, Z., and Han, S. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Zhang, P., Zeng, G., Wang, T., and Lu, W. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pp. 34404–34424, 2023.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Braverman, V., Beidi Chen, and Hu, X. Kivi : Plug-and-play 2bit kv cache quantization with streaming asymmetric quantization. 2023. doi: 10.13140/RG.2.2.28167.37282. URL https://www.researchgate.net/doi/10.13140/RG.2.2.28167.37282.